

Task 1 Bug Report

Severity scale used in this report: Critical, High, Medium, Low. The table below combines all issues discussed during review.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Articles list endpoint returns too many records without enforced pagination	Open GET /api/articles without limit/offset and observe that the server returns the article list directly.	Expected: API should enforce safe pagination defaults or a max page size so one request cannot dump the full dataset. Actual: Endpoint returns data even when no pagination parameters are provided.	High	Pagination is optional but not bounded by a server-side default or maximum limit.
2	Comments endpoint does not paginate large comment threads	Open GET /api/articles/{slug}/comments for an article with many comments.	Expected: Comments should be paginated or loaded in manageable chunks. Actual: All comments can be returned at once, which can become heavy on large threads.	High	Missing server-side pagination or lazy-loading strategy for comments.
3	JWT remains reusable after logout	Log in, log out, then reuse the previously issued JWT in Postman or another client.	Expected: The old token should expire, be revoked, or be rejected after logout. Actual: The previous token still works for protected requests.	High	Logout clears session state on the client, but the backend does not revoke or expire the token.
4	Authentication token is stored in localStorage	Inspect the app storage after login and check where the JWT is saved.	Expected: Prefer HttpOnly, Secure cookies for session tokens when possible. Actual: JWT is saved in localStorage.	High	Auth implementation favors convenient client storage over safer cookie-based handling.
5	Comment form accepts whitespace-only submissions	Open the comment box, enter only spaces, then click Post Comment.	Expected: Whitespace-only comments should be rejected. Actual: A blank-looking comment is accepted and posted.	High	Validation checks for empty strings but does not trim whitespace before submit.
6	Article creation accepts whitespace-only content	Open the article editor, fill required fields with spaces only, then click Publish.	Expected: Fields should be trimmed and rejected if they contain no real text. Actual: An empty-looking article can be published.	High	Frontend and/or backend validation does not normalize whitespace before validation.
7	Sign-up form accepts whitespace-only values	Go to the sign-up page, enter spaces into name, email, and password fields, then submit.	Expected: All sign-up fields should require valid non-whitespace values. Actual: The form can submit successfully with blank-looking inputs.	High	Missing trimming and strict validation on registration fields.
8	Non-existent route returns a confusing error payload	Request a route that does not exist, such as /api/articles/56 when that endpoint is unavailable.	Expected: The API should return a clear 404 Not Found with a consistent error structure. Actual: The response wraps 'not found' inside an error object/array in a confusing way.	Medium	Route-level error handling is inconsistent or not mapped to a proper 404 response.
9	Profile picture setup requires pasting an image URL	Open settings and try to update the profile picture.	Expected: Users should be able to upload or select an image file with preview support. Actual: The UI only asks for a URL, which is not user-friendly for many users.	Medium	Profile avatar flow is URL-only and lacks a file picker/upload component.
10	Article creation flow feels incomplete after publish	Create an article and observe the navigation after submit.	Expected: Users should get a clear success state and be taken directly to the created article page. Actual: The post-publish flow appears delayed or confusing and shows extra UI state/buttons.	Medium	Redirect and success-state handling after create is not polished.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
11	Unexpected focus border appears on the logo / sign-in area	Click the sign-in button or logo and watch the focus styling.	Expected: Focus states should appear only where they help keyboard and accessibility behavior. Actual: A black focus border appears in a visually distracting way.	Low	CSS focus styles are applied too broadly or are not customized for the control.